



**Tecnológico
de Monterrey**

03. Importando datos de archivos

Ciencia de datos para la toma de decisiones I

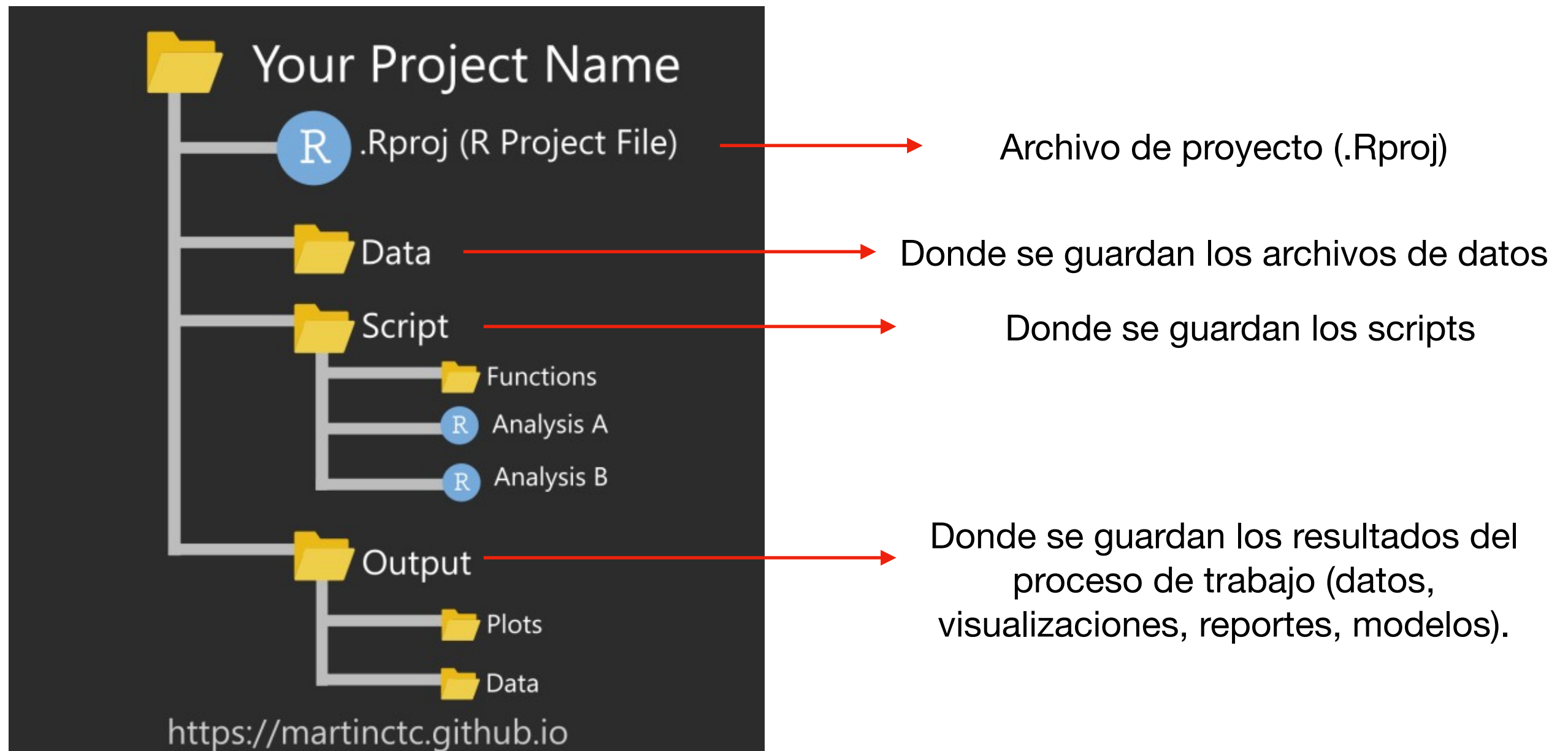
**Jorge
Juvenal
Campos Ferreira**

✉ juvenal.campos@tec.mx

Programa de la sesión de hoy

- Continuar el ejercicio de importación de datos
- Revisar los tipos de variables
- Revisar funciones para trabajar con valores numéricos (estadísticas)

Configuración de un proyecto básico de R



Estructura (sugerida) de un código o script

1. Configuración
2. Carga de librerías
3. Carga de parámetros y funciones propias
4. Carga de datos
5. Limpieza de datos
6. Análisis exploratorio de datos
7. Generación de funciones
8. Generación y pruebas de modelos
9. Generación de gráficas
10. Guardado de archivos

```
# -----  
# Autor: Juvenal Campos  
# Fecha: 2025-08-18  
# Proyecto: Ejemplo para el Tec de Monterrey  
# Descripción: Script dummy que ilustra la estructura básica de un flujo en R.  
# -----  
  
# 1) Configuración -----  
options(digits = 3)  
set.seed(42)  
  
# 2) Carga de librerías -----  
library(dplyr)  
library(readr)  
library(ggplot2)  
  
# 3) Parámetros y funciones propias -----  
params <- list(  
  input = "data/input.csv",      # <- reemplaza si tienes archivo  
  output_dir = "out",  
  target = "mpg"  
)  
paleta_colores <- c("#6950d8", "#3CEAFA", "#00b783",  
                    "#ff6260", "#ffaf84", "#ffbd41")  
if (!dir.exists(params$output_dir)) dir.create(params$output_dir, recursive = TRUE)  
  
clean_names <- function(x){  
  names(x) <- tolower(gsub("[^a-z0-9_]+", "_", names(x)))  
  x  
}  
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))  
  
# 4) Carga de datos -----  
# df <- read_csv(params$input)      # <- real  
# Dummy para ilustrar:  
df <- clean_names(mtcars)
```

Estructura (sugerida) de un código o script

1. Configuración
2. Carga de librerías
3. Carga de parámetros y funciones propias
4. Carga de datos
- 5. Limpieza de datos**
- 6. Análisis exploratorio de datos**
- 7. Generación de funciones**
- 8. Generación y pruebas de modelos**
- 9. Generación de gráficas**
- 10. Guardado de archivos**

```
# 5) Limpieza de datos -----
df <- df %>%
  mutate(across(everything(), \(v) v)) %>%
  filter(!is.na(.data[[params$target]]))

# 6) Análisis exploratorio (EDA) -----
print(summary(df))
print(df %>% summarise(n = n(), p = ncol(.)))

# 7) Generación de funciones -----
# (ya definimos `rmse()` arriba como ejemplo)

# 8) Modelado (dummy) -----
mod <- lm(mpg ~ wt + hp, data = df)
pred <- predict(mod, df)
cat("RMSE:", rmse(df$mpg, pred), "\n")

# 9) Gráficas -----
p <- ggplot(df, aes(wt, mpg)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)

ggsave(file.path(params$output_dir, "scatter.png"), p,
        width = 5, height = 4, dpi = 150)

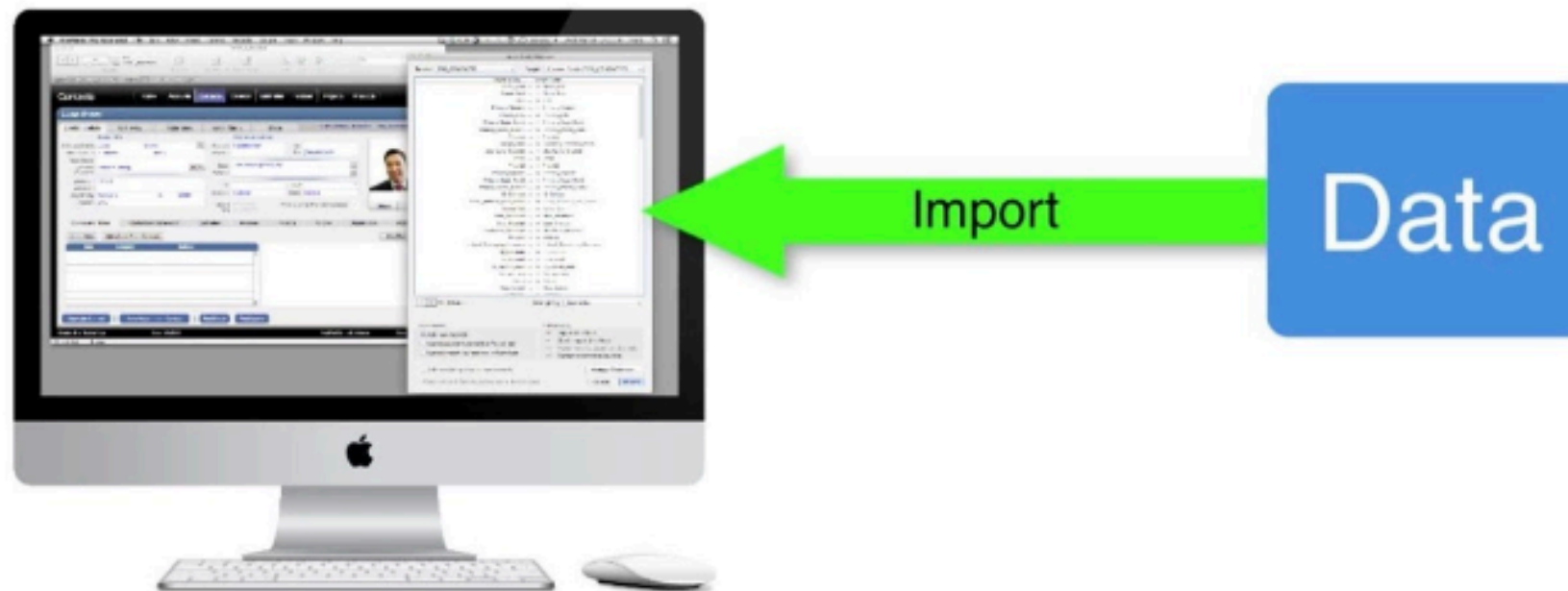
# 10) Guardado de archivos -----
write_csv(df, file.path(params$output_dir, "data_clean.csv"))
saveRDS(mod, file.path(params$output_dir, "model.rds"))
cat("Archivos guardados en:", normalizePath(params$output_dir), "\n")
```

Importar de datos

Importar datos

Cargar datos de un archivo externo a nuestra sesión de R.

IMPORTING DATA



Directorio de trabajo

El directorio de trabajo es la carpeta de la computadora en la cual vamos a estar trabajando.

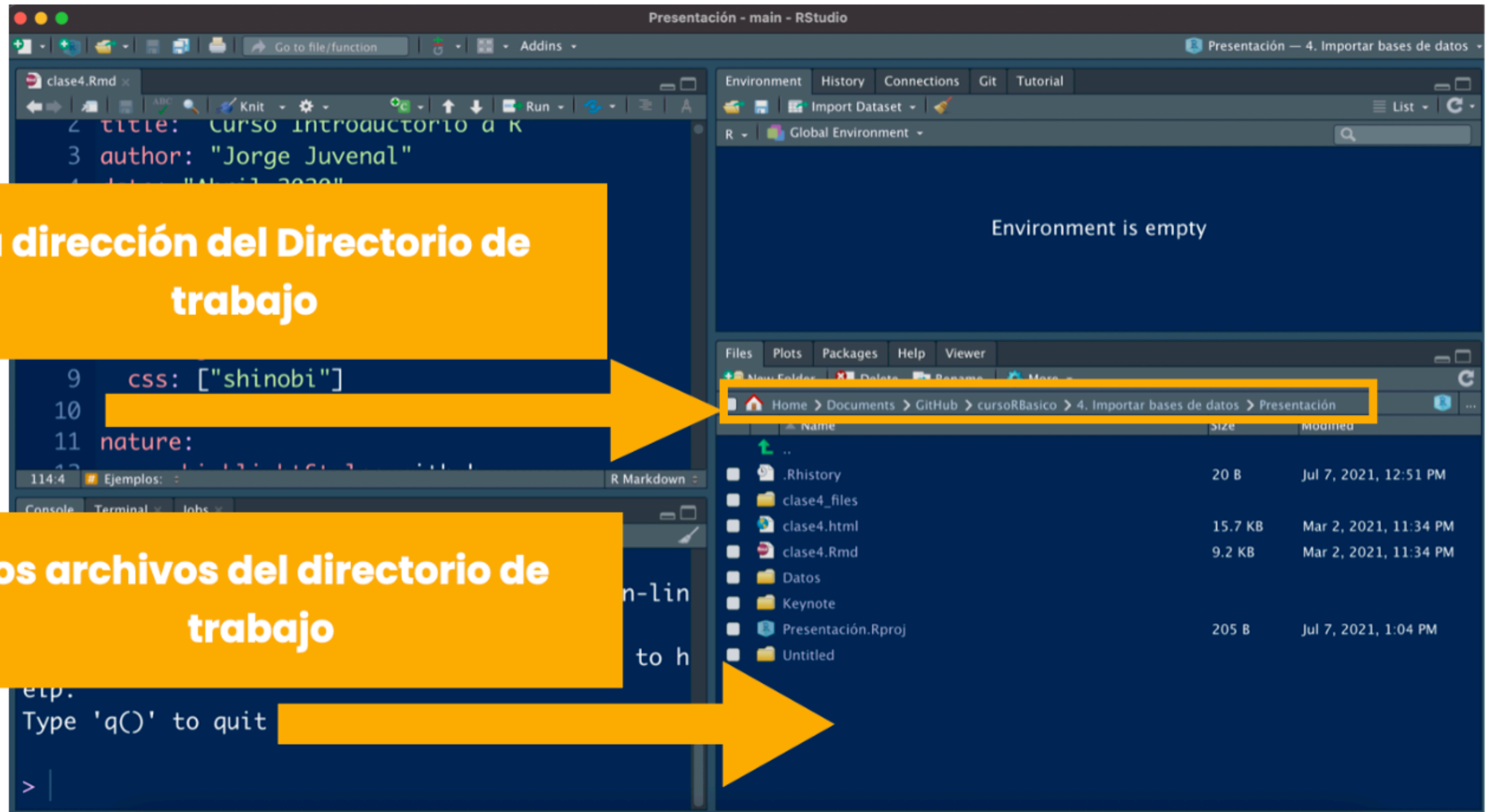
En esta carpeta deben ir (idealmente) todos los archivos que contienen los datos que vamos a utilizar.

En esta carpeta también se van a guardar los archivos que vamos a generar derivados de nuestro análisis (visualizaciones, otras bases de datos, etc.).

Directorio de trabajo

La dirección del Directorio de trabajo

Los archivos del directorio de trabajo



Ubicaciones de un archivo

Para acceder al contenido de un archivo, el programa debe saber donde se ubica.

Podemos acceder a archivos que se encuentren en:

A) Tu computadora

B) En internet

Archivos de internet

Si el archivo se encuentra en internet, una de dos:

a) Lo leemos directamente desde internet.

Hay archivos que se pueden leer desde internet sin necesidad de descargarlos a la compu. Esto depende de la función que usemos, del tipo de archivo y de los permisos de acceso (generalmente solo se puede con archivos de texto plano o archivos únicos).

b) Lo descargamos a nuestra computadora.

Lo descargas y ya. Lo lees desde tu compu. Esto pasa generalmente con excedes, archivos shapefile o archivos *.dta.

Archivos locales

Son los archivos que viven en nuestra computadora.

Para acceder a ellos, necesitamos:

- 1) **La función de R correcta** (y la librería que la contiene).
- 2) **La ubicación** global o la ubicación local de nuestro archivo.

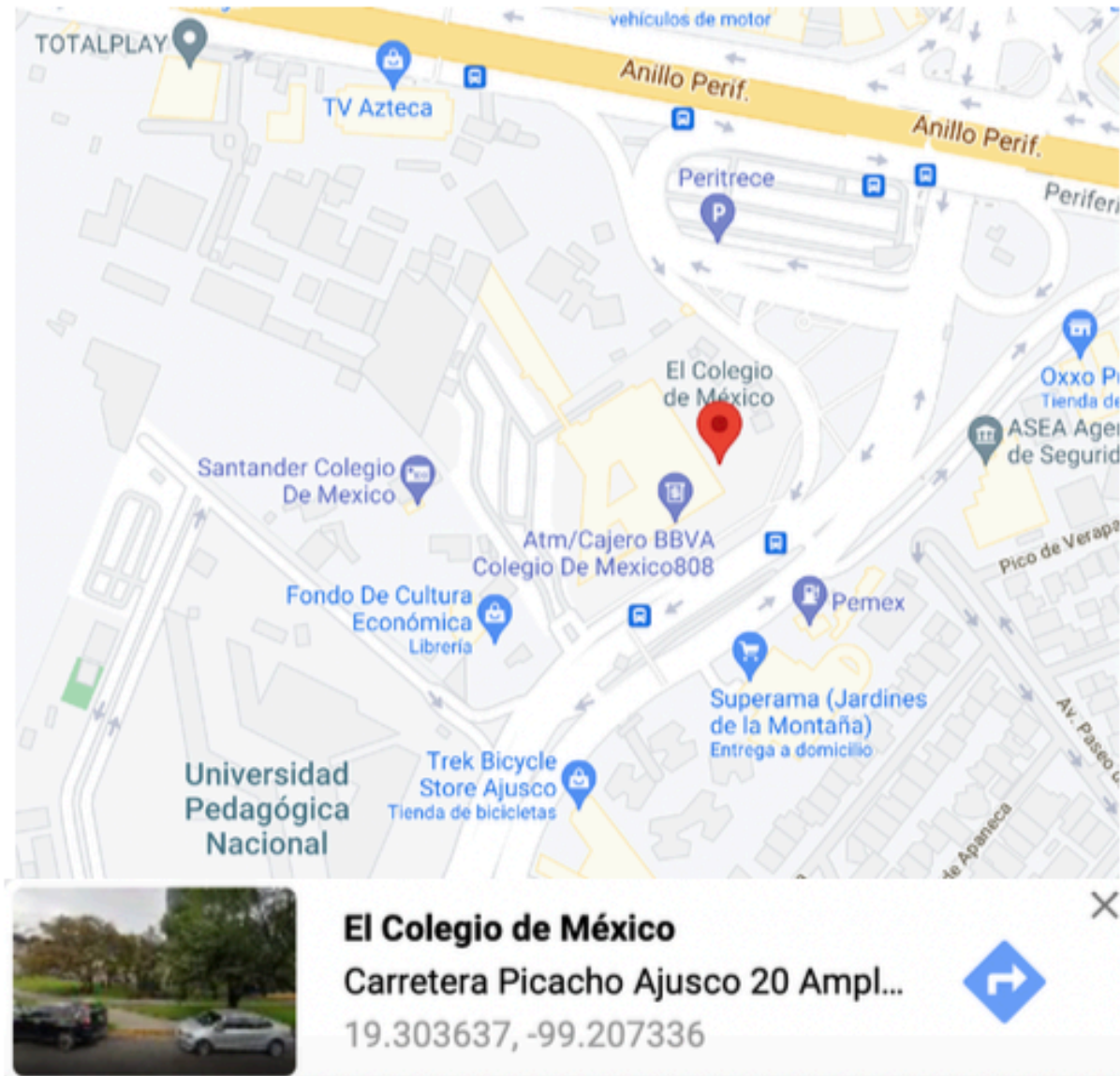
Ruta Global y Ruta Relativa

La ruta global y la ruta local son las ubicaciones de un archivo dentro de nuestra computadora.

La ruta global es la ruta absoluta y única de un archivo en la computadora, y es una forma de identificar el espacio en la memoria que ocupa dicho archivo.

La ruta local es la ruta relativa a otra ubicación en nuestra computadora.

Ruta Global y Ruta Relativa



Ruta absoluta



Ruta relativa

Ruta Global y Ruta Relativa

Las ventajas de usar la ruta absoluta es que siempre va a funcionar en tu computadora. Y ya.

La ventaja de la ruta relativa es que la puedes generar de forma en que **funcione en tu computadora y en las computadoras de otras personas, lo cual mejora la *replicabilidad* de tu análisis y permite compartirlo con otras personas.**

Creando rutas relativas

Hay 3 formas de crear rutas relativas en R:

- 1) Con la función `setwd()`**
- 2) Con la función `here::here()`**
- 3) Utilizando archivos de proyecto de RStudio.**

La que más recomendamos es la opción 3.

Archivo de proyecto (.Rproj)



=



Archivos más utilizados

Los archivos más utilizados para guardar datos son los siguientes:



Información tabular



Objetos de R



Información geográfica



Otros formatos

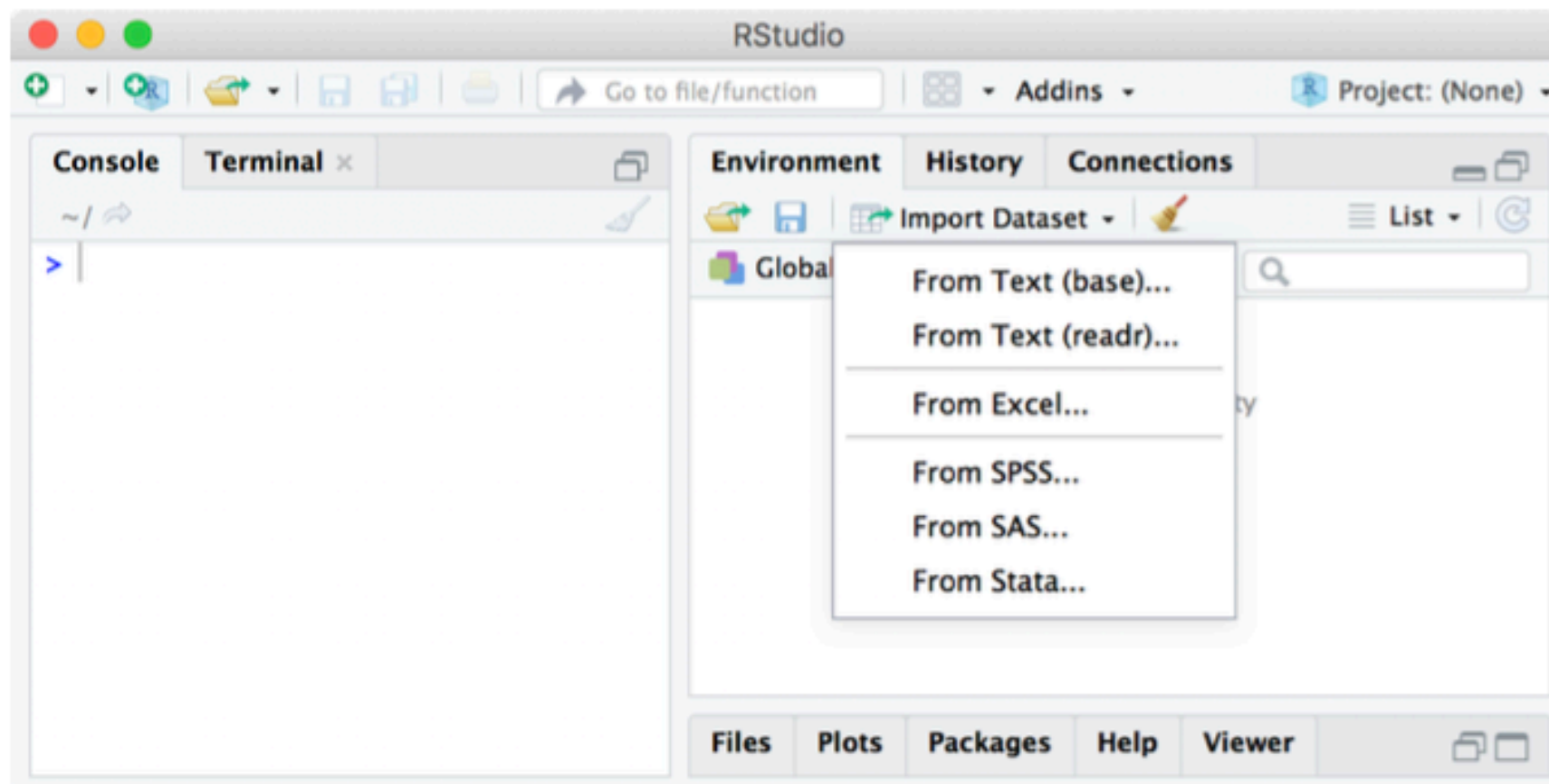
Librerías para importar datos

- `library(readr)`: Archivos texto plano.
- `library(readxl)`: Para excel
- `library(haven)`: Para archivos de Stata, SAS o SPSS
- `library(foreign)`: Para archivos de Stata, SAS o SPSS
- `library(sf)`: Leer archivos geográficos.

Formas de importar datos

Podemos importar archivos de dos maneras:

- 1) Con el **asistente**.*
- 2) Con **código** puro.

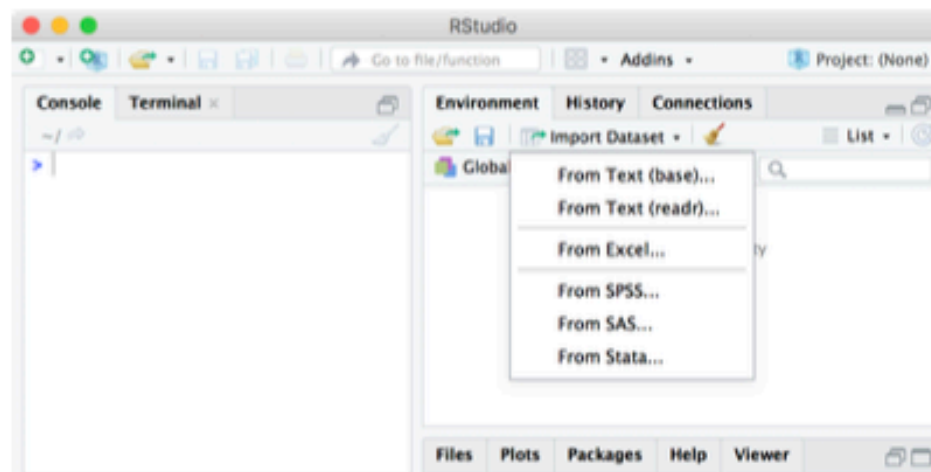


*Sólo para archivos de texto plano, excel, SPSS, Stata y SAS

Formas de importar datos

Método 1: ASISTENTE DE RSTUDIO.

- También puede hacerse el mismo procedimiento a través del Menú:
 - File > Import Dataset > From__ > Browse > Archivo > Import
- **Ventajas:**
 - **Acceso directo al archivo** y R genera la ruta y función correctas
 - Permite **agregar parámetros simples** para la importación (v.g. sheets, skip)
 - **Visualización** de cómo será importado el conjunto de datos
- **Desventajas:**
 - Supone **pasos extra** (copiar y pegar el código) para replicabilidad
 - No permite agregar todos los parámetros (v.g. stringsAsFactors, encoding)



Formas de importar datos

Método 2: SÓLO CON CÓDIGO

- Generando el código directamente:
 - **Requiere cargar las librerías específicas y establecer la ruta del archivo**
- **Ventajas:**
 - Puedes **importar múltiples archivos a la vez** y de todos los formatos posibles
 - **Agregar argumentos específicos** para una importación más sofisticada
 - **Incluir** importación de archivos **dentro de loops o funciones**
 - Si te sabes la función, es **más rápido de escribir**.
- **Desventajas:**
 - Requiere saberte la función exacta
 - Hay que **conocer como es el archivo por dentro**
 - **Pueden ocurrir errores** de los que nos hubiéramos percatado con el otro método.

Problemas de importación

1. Encodings raros.

El **encoding** es el conjunto de símbolos utilizados por un programa para desplegar texto.

Tu sesión de RStudio trabaja por default con un encoding que **podría ser distinto** al que utilizan en otras oficinas/empresas, al que utilizan otros programas o al que usan en otros países.

Por esto mismo, **puede ser necesario el modificar el encoding** utilizado al abrir un archivo o base de datos.

Problemas de importación

2. Errores humanos.

Los más comunes son:

- 1) **Escribiste mal la ruta** del archivo y R no lo encuentra.
- 2) **No conoces bien el archivo** y no te saltaste las líneas necesarias para que se abriera bien.
- 3) **No especificaste la hoja correcta de excel** y te abrió la primera
- 4) No especificaste el **encoding correcto**.

Ejercicio de apertura de datos

- Termine el ejercicio iniciado la clase pasada. Para ello, realice lo siguiente:
 - 1. Vaya al enlace de **CodeShare** que el profesor amablemente le habrá puesto en el pizarrón
 - 2. Descargue el zip que se encuentra en dicho enlace
 - 3. Descomprima el comprimido. **OJO: Si usted usa una computadora Windows, verifique que extrajo los archivos del zip.**
 - 4. Ordene los documentos de dicha carpeta en el orden sugerido en la diapositiva anterior.

Ejercicio de apertura de datos

- 5. **Cargue los datos** de TODOS los archivos disponibles. Para ello, tendrá que hacer lo siguiente:
 - Entender el tipo de archivo que quiere abrir
 - Buscar en internet la función necesaria para abrir el archivo, la librería a la que pertenece dicha función, y descargarla si le hace falta
 - Guardar los datos en un objeto de R, el cual usted va a nombrar de acuerdo al contenido de los datos.
- 6. Una vez terminado, **notifique al profesor**. Aproveche para revisar el resto de diapositivas o para explorar los datos del ejercicio.



Tipos de datos

(vectores atómicos)

Vectores atómicos



Los tipos de datos que podemos almacenar dentro de nuestras estructuras de datos son los siguientes:

Básicos:

- **Character**, para almacenar texto.
- **Numeric**, para almacenar números.
- **Logical**, para almacenar valores lógicos (TRUE/FALSE o T/F).

Compuestos:

- **Factor**, para almacenar datos categóricos.
- **Date**, para almacenar fechas.



Vectores atómicos

- **Character**, para almacenar texto.

```
character <- c("Hola",  
              "Adios",  
              "Buenas tardes",  
              "Buenas Noches")
```

```
character_2 <- c("1", "2",  
                "tres", "cuatro", "5")
```

```
macondo <- c("Muchos años después, frente al pelotón de  
fusilamiento, el coronel Aureliano Buendía había de recordar  
aquella tarde remota en que su padre lo llevó a conocer el hielo.  
Macondo era entonces una aldea de veinte casas de barro y  
cañabrava construidas a la orilla de un río de aguas diáfanas  
que se precipitaban por un lecho de piedras pulidas, blancas y  
enormes como huevos prehistóricos.")
```



Vectores atómicos

- **Numeric**, para almacenar números.

```
# Numéricos
enteros <- c(1L, 2L, 3L, 4L) # Enteros
reales <- c(1,2,3,4) # reales
reales_2 <- c(1e6, 3.1416, 2) # reales con notación científica
```

```
# Operaciones ----

# Suma
c(1,3,4,5) + 5 # [1] 6 8 9 10
# Resta
c(1,3,4,5) - 5 # [1] -4 -2 -1 0
# Multiplicación:
c(1,3,4,5) * 5 # [1] 5 15 20 25
# División:
c(1,3,4,5) / 2 # [1] 0.5 1.5 2.0 2.5
# Exponenciación:
c(1,3,4,5) ^ 2 # [1] 1 9 16 25
```

```
# Secuencias ----
1:100 # Numeros del uno al 100, de uno en uno
seq(from = 0, to = 100, by = 2)
# Numeros del cero al 100, de dos en dos
```




Vectores atómicos

- **Logical**, para almacenar valores lógicos (TRUE/FALSE o T/F).

```
# VALORES LÓGICOS
logical <- c(TRUE, TRUE, TRUE, TRUE, FALSE)
logical_2 <- c(T, T, T, T, F)
# Son lo mismo uno y otro
```

```
# Como resultado de comparaciones:
c(1,2,3,4,5) < 3 # [1] TRUE TRUE FALSE FALSE FALSE
c("a", "b", "c") == "c" # [1] FALSE FALSE TRUE
c("Morelos", "Zacatecas", "Aguascalientes", "CDMX") %in%
  c("Morelos", "Zacatecas") # [1] TRUE TRUE FALSE FALSE
```



Vectores atómicos

- **Factor**, para almacenar datos categóricos.

```
# PASO 1: GENERO UN VECTOR NORMAL
```

```
partidos_ganadores <- c("PAN",  
                        "PRI",  
                        "MORENA",  
                        "MORENA",  
                        "PRI",  
                        "MORENA",  
                        "MORENA",  
                        "MORENA")
```

```
# PASO 2: LO TRANSFORMO A CATEGÓRICO.
```

```
partidos_con_categorias <- factor(partidos_ganadores,  
                                  levels = c("PAN", "PRI", "MORENA"),  
                                  ordered = F)
```

```
> partidos_con_categorias
```

```
[1] PAN    PRI    MORENA MORENA PRI    MORENA MORENA MORENA
```

```
Levels: PAN PRI MORENA
```



Vectores atómicos

- **Factor**, para almacenar datos categóricos.

```
# Generamos el vector de texto
tamanios_playera <-
  c("mediano", "grande", "chico", "chico",
    "mediano", "grande", "grande", "mediano",
    "grande", "chico", "chico", "mediano")

# Sobreescribimos
tamanios_playera <- factor(tamanios_playera,
  levels = c("grande", "mediano", "chico"),
  ordered = T)

tamanios_playera
```

```
[1] mediano grande chico chico mediano grande grande mediano
[9] grande chico chico mediano
Levels: grande < mediano < chico
```

Vectores atómicos



- **Date**, para almacenar fechas.

```
fecha <- Sys.Date()  
fecha  
  
fechas_2 <- as.Date(c("2020-02-02",  
                      "2019-03-04",  
                      "1991-07-08"),  
                    format = "%Y-%m-%d")  
fechas_2
```

```
> fecha <- Sys.Date()  
> fecha  
[1] "2021-06-23"  
> fechas_2 <- as.Date(c("2020-02-02",  
+                        "2019-03-04",  
+                        "1991-07-08"),  
+                        format = "%Y-%m-%d")  
> fechas_2  
[1] "2020-02-02" "2019-03-04" "1991-07-08"
```

Función **class(x)**



La función **class(x)** es la función con la cual podemos saber qué tipo de dato tiene el objeto "x".

```
# Función class(x)
class(fechas_2) # [1] "Date"
class(partidos_con_categorias) # [1] "factor"
class(tamanios_playera) # [1] "ordered" "factor"
class(logical_2) # [1] "logical"
class(vector_1) # [1] "numeric"
class(character_2) # [1] "character"
```

Función **length(x)**



La función **length(x)** nos permite saber el tamaño (# de elementos) que contiene un vector.

```
# Función length(x)
length(fechas_2) # [1] 3
length(partidos_con_categorias) # 8
length(tamamos_playera) # 12
length(logical_2) # [1] 5
length(vector_1) # 5
length(character_2) # 5
```

Coerción.



Supongamos que mezclamos dos o mas tipos de datos:

```
# Tres tipos de datos combinados  
c("Hola", TRUE, 1)
```

Como R no entiende que tipo de dato es el vector, y como necesita que sea de un solo tipo, lo va a transformar al tipo más simple: en este caso, todo lo va a hacer (coercionar) a TEXTO:

```
> c("Hola", TRUE, 1)  
[1] "Hola" "TRUE" "1"
```

Tenemos que tener un solo tipo de datos; si no se cumple esto, R va a transformarlos por nosotros.

Función tibble para armar tablas

Dada una serie de vectores, podemos armar una tabla manualmente con la función **tibble()**

*La librería tibble se carga automáticamente cuando cargamos la librería **tidyverse**

```
library(tibble)

datos <- tibble(
  Nombre      = c("Ana López", "Carlos Ruiz", "Lucía Torres", "Miguel Soto", "Paula Díaz"),
  Edad        = c(32, 45, 28, 37, 41),
  Ciudad      = c("Madrid", "Barcelona", "Valencia", "Sevilla", "Bilbao"),
  Salario      = c(3200, 4100, 2800, 3600, 3900),
  Departamento = c("Marketing", "Finanzas", "Recursos Humanos", "Ingeniería", "Ventas")
)

datos
```

Ejercicio de creación de tablas

- En la carpeta de trabajo anterior, y usando lo visto en la diapositiva previa, recree la siguiente tabla.
- Extra: A través de la coerción, haga que “Nivel_Educativo” sea una variable categórica donde Doctorado > Maestría > Licenciatura y donde “Fecha_Ingreso” sea interpretado como una fecha.

Nombre	Edad	Ingreso_Mensual	Activo	Nivel_Educativo	Fecha_Ingreso
María García	29	3500.5	TRUE	Licenciatura	15/03/21
José Ramírez	43	5200	FALSE	Maestría	22/07/18
Laura Mendoza	35	4100.75	TRUE	Doctorado	03/11/19
Pedro Castillo	26	2800	TRUE	Preparatoria	10/01/23
Sofía Herrera	51	6300.25	FALSE	Licenciatura	28/09/15
Diego Morales	38	4500	TRUE	Maestría	17/06/20

Otras funciones

Función Secuencia

La función secuencia nos permite generar secuencias de números.

Esto es útil cuando necesitamos una serie de valores a intervalos conocidos y no queremos generarlos uno por uno.

```
# Secuencia de números del 0 al 100 de cinco en cinco:
```

```
seq(from = 0, to = 100, by = 5)
```

```
# Secuencia de numeros del 0 al 1 en intervalos de 0.1
```

```
seq(from = 0, to = 1, by = 0.1)
```

```
# Secuencia del 345678 al 345900 de 32 en 32
```

```
seq(345678, 345900, 32)
```

```
# Alternativa: la secuencia de uno en uno se genera con el símbolo ":"
```

```
1:32 # Todos los números del 1 al 32
```

Acceso a datos dentro de una tabla

Suponga la siguiente tabla:

```
library(tibble)
tabla <- tibble(id = c(1,2,3,4,5,6),
  nombre = c("Juan", "Pedro", "María",
    "Alejandra", "Patricia", "Pablo"),
  sexo = c("H", "H", "M", "M", "M", "H"),
  calif = c(100, 100, 90, 80, 90, 100))
```



A tibble: 6 × 4

id	nombre	sexo	calif
<dbl>	<chr>	<chr>	<dbl>
1	Juan	H	100
2	Pedro	H	100
3	María	M	90
4	Alejandra	M	80
5	Patricia	M	90
6	Pablo	H	100

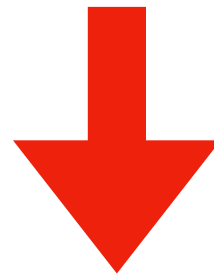
Acceso a datos dentro de una tabla

```
A tibble: 6 × 4
  id nombre  sexo  calif
<dbl> <chr>   <chr> <dbl>
1  1 Juan    H     100
2  2 Pedro  H     100
3  3 María  M      90
4  4 Alejandra M      80
5  5 Patricia M      90
6  6 Pablo  H     100
```

```
# Acceder a los valores de calificación:
tabla$calif
```

```
# Acceder a los valores de sexo:
tabla$sexo
```

```
# Acceder a los valores de nombre:
tabla$nombre
```



Accediendo a estos valores ya se pueden utilizar para hacer cálculos básicos.

```
> tabla$calif
[1] 100 100 90 80 90 100
> tabla$sexo
[1] "H" "H" "M" "M" "M" "H"
> tabla$nombre
[1] "Juan"      "Pedro"      "María"      "Alejandra" "Patricia" "Pablo"
```

Estadística básica y descriptiva

Sea x un vector con números (numeric o integer), las funciones básicas para estadística descriptiva son las siguientes:

Tendencia central:

`mean(x)` — Media

`median(x)` — Mediana

Dispersión:

`var(x)` — Varianza

`sd(x)` — Desviación estándar

`IQR(x)` — Rango intercuartílico

Extremos y posición:

`min(x)` — Valor mínimo

`max(x)` — Valor máximo

`range(x)` — Rango (Valor mínimo y valor máximo)

`quantile(x)` — Cuantiles

Resumen general:

`summary(x)` — Min, Q1, mediana, media, Q3, Max

`length(x)` — N° de elementos

Forma de la distribución:

`moments::skewness(x)` — Asimetría

`moments::kurtosis(x)` — Curtosis

Tip:

Usa `na.rm = TRUE` si hay valores faltantes.

Ejercicio de cálculos básicos

- Con la tabla que generó previamente, obtenga todos los estadísticos mencionados en la diapositiva anterior.

Nombre	Edad	Ingreso_Mensual	Activo	Nivel_Educativo	Fecha_Ingreso
María García	29	3500.5	TRUE	Licenciatura	15/03/21
José Ramírez	43	5200	FALSE	Maestría	22/07/18
Laura Mendoza	35	4100.75	TRUE	Doctorado	03/11/19
Pedro Castillo	26	2800	TRUE	Preparatoria	10/01/23
Sofía Herrera	51	6300.25	FALSE	Licenciatura	28/09/15
Diego Morales	38	4500	TRUE	Maestría	17/06/20

Librería {*Datapasta*}

Librería que permite copiar y pegar tablas de fuera a nuestra sesión de R.

Ventajas: Nos ahorra tiempo de recolección y carga de datos.

Desventajas:

- * Scripts mas largos y desordenados
- * Los datos están codeados en el código.
- * Solo es práctico para tablas *chicas*



Instalar

```
DATAPASTA
```

```
Paste as tribble
```

```
Paste as vector
```

```
Paste as vector (vertical)
```

```
Paste as data.frame
```



Archivos nativos de R

Al trabajar con R y RStudio, generamos cuatro tipos de archivos nativos de R, estos son los siguientes:

- *.Rproj,
- *.RData,
- *.R y
- *.rds.

Los scripts son aquellos archivos que terminan en *.r

Fin de la clase